

Difinity Echo: Consolidated Product, Architecture and FYP Report

1. Executive Summary

Difinity Echo is a **continuous AI engineering analytics platform** designed to help engineering leaders understand how AI-assisted software development relates to engineering spend, delivery performance, software quality, support demand, and product or customer outcomes.

Its central purpose is not simply to measure AI-tool adoption. Most organisations can already obtain basic statistics such as active users, tokens consumed, licence utilisation, or provider usage. Echo addresses the more difficult question:

What measurable engineering and organisational outcomes are associated with the organisation's investment in AI-assisted development?

Echo connects evidence from AI coding tools with engineering systems of record such as source control, work management, deployments, production monitoring, customer support, and product analytics.

The intended evidence chain is not a rigid linear pipeline. It is a **many-to-many evidence graph** connecting AI activity to accepted changes, commits, pull requests, work items, deployments, quality events, support demand, and product outcomes.

A simplified representation is:

AI activity → accepted edit/change set → commit → pull request

with additional relationships such as:

Pull request ↔ work item

Work item ↔ initiative/product/configured organisational entity

Commit → build → artifact → production deployment

Deployment → quality, support and product-outcome observation windows

AI expenditure and contribution evidence are attached at the appropriate points in this graph.

The platform is therefore intended to provide an explainable, traceable analytical view rather than a single productivity score or simplistic ROI calculation.

For the Final Year Project, the objective is to design and implement a working prototype of this platform, starting with synthetic data, selected connectors, a canonical data model, AI telemetry collection, an ingestion and analytics pipeline, correlation logic, metrics, dashboards, and staged validation.

2. Product Definition

2.1 What Echo Is

Echo is Difinity's **continuous AI engineering analytics platform**.

Its purpose is to connect:

- AI-tool usage
- AI expenditure
- engineering work
- delivery performance
- software quality
- production reliability
- support demand
- product outcomes
- organisational context

The primary intended users are:

- CTOs
- VP Engineering
- Heads of Engineering
- engineering leadership
- product and programme stakeholders
- selected engineering teams
- organisational leadership

The platform allows these stakeholders to examine whether **AI-linked engineering work behaves differently from comparable work where no AI contribution was observed**.

The product's marketing promise is ROI visibility:

Make the real ROI of engineering AI visible.

However, ROI is not intended to be a single magic metric. The actual product is an analytics and evidence platform exposing the underlying spend, delivery, quality, and outcome evidence from which supported conclusions may be made.

2.2 Core Product Messaging

The approved product headline is:

See how AI changes delivery speed, software quality and product outcomes.

The supporting message is:

Every AI-assisted change leaves an echo across engineering. Echo shows where it improves delivery, where it creates rework and what reaches the customer.

Echo therefore aims to move organisations from:

AI adoption reporting

to:

AI engineering outcome analysis.

3. The Organisational Problem

Organisations are increasingly purchasing AI coding assistants such as Claude Code, OpenAI Codex, GitHub Copilot, and Cursor.

However, purchasing licences and observing that developers use these tools does not demonstrate whether the expenditure produces engineering value.

The engineering lifecycle is fragmented across multiple systems.

For example:

Engineering information	Typical source
AI sessions and usage	Claude Code, Codex, Copilot, Cursor
AI usage/cost information	Provider administration APIs
Commits and pull requests	GitHub, Bitbucket
Tickets and planned work	Jira, Linear
Builds and deployments	GitHub Actions, Bitbucket Pipelines
Production errors	Sentry
Customer support	Zendesk, Intercom
Product usage/outcomes	Amplitude, PostHog

Individual platforms can explain their own domain, but they generally do not establish the complete cross-system relationship.

Consequently, organisations struggle to answer questions such as:

- How much are we spending on engineering AI?
- Which products, teams, initiatives or applications account for that spend?
- Which shipped changes actually contain observed AI contribution?
- Is AI-linked work delivered faster?
- Does it require more or less review effort?

- Is it associated with increased rework?
- Are AI-assisted releases associated with more defects or incidents?
- Does support demand change after AI-linked releases?
- What customer or product outcomes follow?
- Is the underlying evidence sufficiently complete to trust the comparison?

The main research and engineering problem is therefore **correlation and evidence quality across heterogeneous engineering systems**, rather than simple data collection.

4. What Echo Is Not

Echo must not be presented as:

- a developer surveillance system;
- an employee-ranking system;
- a generic engineering observability product;
- a generic cost dashboard;
- a replacement for Jira, Linear, GitHub, Bitbucket, Sentry, Zendesk, Intercom, Amplitude, or PostHog;
- a one-time consultancy report;
- a board investment-approval workflow;
- a system that calls activity counts productivity;
- a system that assumes correlation proves causality;
- a platform that reduces engineering productivity to a single score.

Developer- and session-level metrics may exist, but they are primarily intended as **diagnostic drill-down information**, not employee-performance rankings.

5. Relationship with Other Difinity Products

Difinity's broader product structure currently consists of three conceptual layers:

Difinity Hub

Governance and configuration layer.

Difinity Flow

Runtime gateway and enforcement layer.

Difinity Echo

Continuous analytics layer.

Echo should operate as a **standalone platform**.

Where Hub and Flow are deployed, their events may enrich Echo's analytical evidence.

6. Core Analytical Model

6.1 Canonical Evidence Model

Echo maintains a stable set of canonical engineering facts.

Important facts include:

- AI session
- developer interaction
- AI usage event
- AI cost event
- accepted edit or change set
- commit
- pull request
- work item
- build
- artifact
- deployment or release
- quality signal
- support signal
- product/customer outcome measurement

A key design principle is that these entities should not be represented as one rigid hierarchy.

For example, a work item is not necessarily a causal stage between AI activity and code.

Instead, it is a relationship:

Pull request ↔ work item

Similarly, initiatives, products, teams, repositories, services, and cost centres can overlap.

The resulting architecture is better represented as an **evidence graph** rather than a simple sequence of database tables.

7. Customer-Configured Dimensional Graph

The FYP proposal originally describes a logical progression such as:

**AI coding session → code contribution → pull request → ticket → epic → initiative → objective/
outcome**

This is useful as a simplified explanation of traceability.

However, the current Echo definition takes a more flexible approach.

Echo must **not hardcode one organisational hierarchy** such as:

Company → Organisation → Team → Initiative → Epic → Ticket.

Real organisations have different structures.

One customer may use:

Company → Engineering Organisation → Team

and:

Portfolio → Initiative → Epic → Ticket.

Another may use:

Company → Division → Tribe → Squad

and:

Program → Workstream → Initiative.

A third may organise primarily around:

Product → Service → Component.

Therefore, Echo uses customer-configured dimension families.

7.1 Initial Dimension Families

Dimension family	Example levels
Organisation	Company, division, department, tribe, squad, team
Work	Portfolio, programme, initiative, workstream, epic, feature, ticket
Product	Product, application, service, component, repository
Financial	Cost centre, budget, contract, vendor
People	Role, location, employment type, individual

These are templates rather than mandatory levels.

8. Relationship Semantics

Cross-hierarchy relationships should be explicit and may be many-to-many.

Initial relationship types include:

- contains
- owns
- contributes to
- belongs to
- delivers
- operates
- funded by
- affects

Each relationship should preserve information including:

- source
- relationship role
- effective period
- mapping method
- evidence state
- rule version
- optional allocation weight

The platform must also detect data-configuration problems such as:

- graph cycles
- orphan entities
- conflicting ownership
- unresolved mappings
- contradictory relationships
- allocation weights that do not reconcile

Likely configuration inputs include:

- HRIS or organisational directory data
- Jira
- Linear
- GitHub
- Bitbucket
- CODEOWNERS
- service catalogues
- procurement systems
- financial systems
- audited manual overrides

9. AI Evidence Cohorts

One of Echo's most important analytical rules is that AI contribution cannot be represented using a simple binary flag.

Four distinct cohorts are maintained.

9.1 AI-Linked Work

There is direct or manually confirmed evidence connecting AI activity to a shipped engineering change.

This is the strongest AI evidence category.

9.2 AI-Exposed Only

An AI session occurred near or during the relevant work, but Echo cannot establish that the AI contributed to the shipped change.

This must not automatically be treated as AI-linked work.

9.3 No AI Observed, Capture-Complete

Telemetry was healthy and complete enough to determine that no qualifying AI contribution was observed.

This cohort forms the valid comparison population for AI-versus-non-AI analysis.

9.4 AI Status Unknown

Collection, identity resolution, or linkage was incomplete.

This work is excluded from direct AI-versus-non-AI comparisons.

A fundamental analytical rule is therefore:

Unknown must never silently become non-AI.

10. Measurement and Aggregation Contract

Different engineering metrics exist at different natural levels.

Echo therefore calculates each metric at its valid **natural grain**.

Metric family	Natural grain
Session duration	AI session
Interaction intensity	AI session
AI spend	Cost event

Metric family	Natural grain
Review duration	Pull request
Review rework	Pull request
Work cycle time	Work item
Change-failure rate	Deployment
Rollback rate	Deployment
Escaped defects	Mature release window
Incident rate	Mature release window
Support rate	Release/feature-exposed customer cohort
Product outcome	Declared product, feature, initiative or exposure unit

Every metric definition should identify:

- natural grain
- eligible population
- numerator
- denominator
- comparison cohort
- AI cohort
- reporting window
- observation window
- permitted dimensions
- deduplication key
- allocation treatment
- minimum sample size
- maturity requirement
- evidence state
- known limitations

11. Roll-Up and Aggregation Rules

Echo must avoid statistically invalid dashboard calculations.

Key rules include:

- Rates are recalculated from the underlying numerator and denominator.
- Medians and percentiles are recomputed from eligible observations or mergeable statistical sketches.
- Child-level medians are not averaged.
- AI-versus-non-AI differences calculated at lower organisational levels are not simply averaged.
- Company totals are deduplicated using underlying fact identifiers.
- Incomplete or very small cohorts may be suppressed.
- Comparisons are recalculated at the selected organisational or product scope.

- Dashboard metrics expose their denominator, observation period, maturity, and evidence health.

These rules are important because seemingly simple averaging can produce misleading engineering conclusions.

12. Ownership, Contribution and Overlapping Work

Engineering work often crosses organisational boundaries.

Echo therefore distinguishes several scope semantics.

Owned Work

Work for which the selected entity is the recognised owner.

Contributed Work

Work to which the entity made a contribution even though it may not own it.

All Involvement

All work in which the entity had any relevant involvement.

Intersection

Work satisfying multiple selected dimensions simultaneously.

For example:

- View by Organisation may show all initiatives an organisation owns or contributes to.
 - Filtering to Payments and viewing by Initiative shows initiatives to which Payments contributed.
 - Selecting an initiative and grouping by Team shows participating teams.
-

13. Attributed Versus Touched Measures

Echo also distinguishes between **attributed** and **touched** data.

Attributed

A value is fractionally allocated so that organisational totals reconcile.

Potential policies include:

- primary owner

- explicit percentage
- proportional contribution
- equal split
- unallocated

Touched

The full value or event is shown against every entity that touched it.

Touched metrics are useful for investigation but are **non-additive**.

Echo must not silently choose the allocation method.

The default executive view should use attributed measures, while touched measures should clearly state that totals are non-additive.

14. Core Metric Catalogue

14.1 Investment and Adoption

Important metrics include:

- total AI spend by cost class;
- spend per active developer;
- spend by provider;
- spend by tool;
- spend by product;
- spend by organisation;
- spend by configured work entity;
- AI-active developers;
- AI-linked share of eligible shipped work;
- unallocated licence cost;
- unallocated usage cost.

14.2 Developer and Session Diagnostics

Possible diagnostic measures include:

- mean session duration;
- median session duration;
- mean interaction intensity;
- median interaction intensity;
- human messages per session;
- agent tool actions per human message;
- autonomous execution duration;
- model usage;

- tool usage;
- harness usage;
- permission-mode usage.

These are specifically **not productivity scores**.

15. Delivery Metrics

Potential delivery measures include:

- work-item cycle time;
- change lead time;
- branch creation to pull-request opening;
- review request to approval;
- review request to merge;
- review rounds;
- additional commits following review;
- deployment frequency per service-week;
- feature idea-to-release time;
- initiative idea-to-release time.

These measures can be compared between eligible AI-linked and capture-complete no-AI-observed cohorts.

16. Quality Metrics

Echo's quality analysis may include:

- escaped-defect rate;
- change-failure rate;
- rollback rate;
- revert rate;
- hotfix rate;
- incident count by severity;
- mean time to recovery;
- review comments;
- post-review code churn;
- newly introduced high-severity security findings;
- newly introduced critical security findings;
- CI failure rate;
- CI rerun rate;
- post-release error-budget burn.

The purpose is to prevent organisations from interpreting faster coding or delivery as a productivity improvement if the resulting work creates greater review, rework, reliability, or support costs.

17. Customer Support and Product Outcomes

Downstream customer-facing measures may include:

- support cases per 1,000 exposed accounts;
- affected customer accounts;
- affected product areas;
- feature adoption;
- task completion;
- conversion;
- retention;
- reliability;
- customer-defined initiative metrics;
- customer-defined product metrics.

An important analytical limitation is that product outcomes must be attached at the valid feature, product, release, initiative, or exposure level.

They should **not be attributed directly to one developer or pull request unless an appropriate mapping genuinely supports that claim.**

18. Cost and ROI Treatment

Echo separates multiple categories of AI cost.

These include:

- provider-metered usage;
- invoiced licence cost;
- allocated licence cost;
- estimated or modelled cost;
- implementation cost;
- enablement cost;
- infrastructure cost;
- verification cost.

These values must reconcile with the relevant in-scope financial ledger.

Each allocation should make its basis visible.

A major principle is:

Linked cost is not automatically ROI.

Echo should only present a supported ROI conclusion when:

- AI expenditure is measured or transparently allocated;
- delivery linkage is sufficiently complete;

- quality and outcome windows have matured;
- AI contribution uses a defined analytical method;
- the customer accepts the financial or operational value model.

Otherwise, the result should be labelled appropriately, for example:

- Supported association
- Provisional
- Unmeasured
- ROI unavailable

Another governing principle is:

Missing evidence is not negative evidence.

19. Initial Integration Stack

The initial connectors are an **80/20 product-value selection** rather than an attempt to integrate every engineering product immediately.

19.1 AI Coding Tools

First-class initial adapters:

1. Claude Code
 2. OpenAI Codex
 3. GitHub Copilot
 4. Cursor
-

19.2 Work Tracking

- Jira Cloud
 - Linear
-

19.3 Source Control

- GitHub Cloud
 - Bitbucket Cloud
-

19.4 Deployment

- GitHub Actions
- GitHub Deployments
- Bitbucket Pipelines

- Bitbucket Deployments

A provider-neutral signed deployment-event API should also exist for systems such as:

- Jenkins
- CircleCI
- Argo CD
- Buildkite
- Vercel
- other deployment platforms

A successful CI build or merged pull request must **never be assumed to represent a production deployment**.

19.5 Production Quality

- Sentry Cloud
-

19.6 Customer Support

- Zendesk Support
 - Intercom
-

19.7 Product Analytics

- Amplitude
 - PostHog
-

20. Candidate End-to-End Customer Stacks

Three coherent initial configurations have been identified.

Stack 1

Jira

→ GitHub

→ GitHub Actions and Deployments

→ Sentry

→ Zendesk

→ Amplitude

Stack 2

Linear

→ GitHub

→ GitHub Actions and Deployments

→ Sentry

→ Intercom

→ PostHog

Stack 3

Jira

→ Bitbucket

→ Bitbucket Pipelines and Deployments

→ Sentry

→ Zendesk

→ Amplitude

The actual first implementation should follow the technology stack used by the **first funded design partner** rather than attempting to build all three simultaneously.

21. Endpoint Collection Architecture

AI telemetry is collected through vendor-specific adapters while maintaining a provider-neutral internal model.

The current collection path is:

Vendor hooks / OpenTelemetry / administration APIs

↓

Vendor adapter

↓

Local allowlist privacy processor

↓

Session and event normaliser

↓

Local Git provenance linker

↓

Encrypted durable buffer



Echo ingestion API



Evidence graph and analytics

Endpoint telemetry and provider administration data serve different purposes.

Endpoint Collection Provides

- session information;
- tool usage;
- permission information;
- local change context;
- repository context;
- Git provenance.

Provider Administration APIs Provide

- usage reconciliation;
- seats;
- tokens or credits;
- provider-level spending.

Source Control Provides

- commits;
- pull requests;
- repository relationships.

Downstream Connectors Provide

- deployments;
- quality signals;
- support signals;
- customer and product outcomes.

22. Intended Telemetry Metadata

The metadata sent to Echo may include:

- pseudonymous customer-controlled identity;
- AI provider;
- AI tool;
- AI model;

- agent;
 - session identifier;
 - turn identifier;
 - timestamps;
 - duration;
 - token metadata;
 - cost metadata;
 - tool category;
 - tool outcome;
 - tool duration;
 - permission mode;
 - permission decision;
 - repository references;
 - commit references;
 - pull-request references;
 - collector version;
 - schema version;
 - health state;
 - linkage method;
 - linkage confidence;
 - rule version.
-

23. Privacy Boundary

Privacy is a core architectural requirement.

Echo's default collection model is **metadata-only**.

The following information is prohibited from being transmitted by default:

- prompts;
- AI responses;
- reasoning or thought content;
- source code;
- raw diffs;
- raw file paths;
- raw branch names beyond a locally extracted work key;
- raw commands;
- tool arguments;
- tool results;
- error-message text;
- secrets;
- environment-variable values;
- clipboard contents;
- screenshots;
- keystrokes;
- full transcripts.

Sanitisation occurs **locally**, before information is persisted into the outbound queue or buffer.

Native content-bearing vendor payloads should therefore not be forwarded directly into Echo Cloud.

24. Cloud Architecture

The current working architectural decision uses four main storage and processing technologies.

24.1 Kafka

Kafka functions as the:

- durable event log;
- decoupling layer;
- replay mechanism;
- burst-absorption mechanism;
- failure-isolation mechanism.

Kafka is retained primarily for scalability and resilience.

Its presence does not mean every dashboard metric must update within seconds.

24.2 ClickHouse

ClickHouse is the preferred **hot analytical datastore**.

It is intended for:

- high-cardinality analytics;
- engineering traces;
- AI and non-AI cohorts;
- percentile calculations;
- large-scale time-based aggregation;
- interactive dashboards.

However, ClickHouse has **not yet been benchmarked using production-scale Echo-shaped data**.

This remains an explicit validation requirement.

24.3 S3-Compatible Object Storage

S3 or a qualified S3-compatible system acts as the canonical event archive.

It provides:

- durable historical storage;
- replay;

- recomputation;
 - recovery;
 - long-term data retention.
-

24.4 PostgreSQL

PostgreSQL acts as the mutable **control plane**, rather than the main event warehouse.

It stores information such as:

- tenants;
 - users;
 - roles;
 - enrolments;
 - connector configuration;
 - retention settings;
 - identity mappings;
 - ingestion-job state;
 - audit metadata.
-

25. End-to-End Event Flow

The current cloud event flow is:

Echo Edge + server-side connectors

↓

Regional authenticated admission API

↓

Kafka event log

↓

ClickHouse writer

↓

ClickHouse hot analytics

↓

Tenant-aware Query API

↓

Overview / Compare / Explore / Live Trace / Data Health

In parallel:

Kafka

↓

Archive writer

↓

S3-compatible canonical archive

↓

Replay and recomputation

↓

ClickHouse

PostgreSQL provides policy and control-plane information to admission, identity, and query services.

26. Freshness Model

Echo continuously processes data and supports near-real-time analysis where technically appropriate.

However:

second-level freshness is not a core product promise.

Each source and metric should expose:

- event watermark;
- source freshness;
- maturity state;
- late-event behaviour;
- observation window.

For strategic engineering analytics, a result becoming available after several minutes or hours may still satisfy the user requirement.

27. Dashboard Information Architecture

The current Echo direction is organised around six main product areas.

27.1 Overview

The Overview page presents a configurable **View By** selector.

Possible dimensions include:

- organisation;
- team;
- initiative;
- product;
- application;
- service;
- cost centre;
- another tenant-defined level.

Filters remain independent and may include:

- time window;
- hierarchy level;
- application;
- repository;
- AI provider;
- AI tool;
- work type;
- evidence state.

28. Consolidated Comparison Matrix

Rather than maintaining a separate dashboard table for every metric, the primary Overview should present a single consolidated comparison matrix.

Recommended structure:

Column	Purpose
Entity	Current group-by member
AI Spend	Attributed AI expenditure
AI Coverage	Eligible shipped work with qualifying AI contribution
Delivery	AI-linked versus no-AI-observed comparison
Quality	AI-linked versus no-AI-observed comparison

Column	Purpose
Customer Outcome	Valid customer/product comparison
Evidence	Coverage, maturity and sample health

Each analytical family has one selected primary metric.

Initial recommendations are:

- **Investment:** attributed AI spend
- **Adoption:** AI-linked share of eligible shipped work
- **Delivery:** work-item cycle time
- **Quality:** escaped-defect rate
- **Customer:** declared product outcome or support rate
- **Evidence:** linkage coverage and maturity

Users may switch the metric used within a family.

For example, Delivery could switch from work-item cycle time to:

- review time;
- change lead time;
- deployment frequency.

There is currently **no requirement for a generated executive narrative** and **no requirement for a composite productivity or ROI score**.

29. Compare

The Compare page provides deeper AI cohort analysis.

It may present:

- AI-linked distributions;
 - no-AI-observed distributions;
 - exact metric values;
 - percentage or absolute differences;
 - matched-cohort methodology;
 - sample sizes;
 - trends over time;
 - uncertainty or evidence strength;
 - owned versus contributed scope;
 - attributed versus touched treatment.
-

30. Explore

Explore allows users to drill into configured organisational and delivery entities such as:

- organisation;
 - team;
 - initiative;
 - product;
 - service;
 - application;
 - cost centre;
 - intersections between these entities.
-

31. Live Trace

Live Trace is one of the most important explainability capabilities.

A user should be able to search for a:

- ticket;
- pull request;
- deployment;

and inspect its connected:

- AI evidence;
- engineering work;
- release evidence;
- quality signals;
- support signals;
- product outcomes.

Each relationship should expose an evidence status such as:

- direct;
- confirmed;
- inferred;
- conflicted;
- missing.

This provides the auditability required to inspect how an aggregate dashboard result was constructed.

32. Data Health

Data Health is a first-class product capability rather than a purely administrative screen.

It monitors:

- connector availability;
- connector freshness;
- event-to-query latency;
- source completeness;
- AI-status coverage;
- identity resolution;
- linkage coverage by relationship type;
- cost reconciliation;
- hierarchy mapping;
- ownership conflicts;
- allocation reconciliation;
- orphan entities;
- mature versus incomplete quality windows;
- mature versus incomplete product-outcome windows.

This is essential because Echo must tell users not only what a metric says, but also whether enough evidence exists for that metric to be trusted.

33. Data Policy and Configuration

This section includes controls for:

- collector policies;
 - identity visibility;
 - roles;
 - access;
 - retention;
 - individual-level reporting;
 - organisational hierarchies;
 - work hierarchies;
 - relationship mappings;
 - allocation rules;
 - correction history;
 - dispute history;
 - audit records.
-

34. Tenant Isolation

34.1 Standard SaaS

Standard SaaS deployment should use:

- bounded regional cells;
- authenticated tenant enrolment;
- tenant identity attached to every fact and relationship;

- tenant-aware joins;
- tenant-aware aggregates;
- tenant-aware caches;
- tenant-specific object storage controls;
- read-only tenant-specific ClickHouse identities;
- ClickHouse row policies;
- PostgreSQL row-level security;
- cross-tenant negative testing;
- tenant quotas.

Browser and BI clients should not receive direct access to ClickHouse.

ClickHouse row policies are treated as **defence in depth**, not as a physical isolation boundary.

35. Dedicated Enterprise Deployment

Customers requiring stronger isolation may receive dedicated:

- cloud account;
- network;
- Kafka cluster;
- ClickHouse environment;
- object storage;
- PostgreSQL;
- encryption keys;
- secrets;
- backups.

A schema or database inside a shared cluster should not be marketed as full physical isolation.

36. Deployment Portability

Echo is intended to support:

- Difinity-managed SaaS on AWS;
- customer-owned AWS accounts;
- connected private data centres;
- air-gapped infrastructure.

Core technology contracts should remain portable.

These include:

- Kafka
- ClickHouse SQL
- PostgreSQL
- S3 API

- Parquet
- OCI containers
- Helm
- OIDC
- SAML

AWS-native services may implement parts of these contracts, but should not become mandatory authoritative domain dependencies.

37. Recommended Development Sequence

The current preferred delivery order is:

1. Canonical event schema, privacy registries and testing framework.
2. Minimal Claude Code, Codex, Copilot and Cursor adapters.
3. Local Git provenance and cloud ingestion.
4. Provider cost and identity reconciliation.
5. One complete end-to-end stack using Jira or Linear with GitHub and GitHub Deployments.
6. Live Trace and Data Health.
7. Customer-configurable dimensional graph and mapping health.
8. Overview comparison matrix.
9. Sentry plus Zendesk or Intercom.
10. Amplitude or PostHog.
11. Bitbucket integration path.
12. More advanced analytical methods and additional connectors according to customer demand.

This sequence is a working recommendation rather than a fixed commercial release plan.

The actual initial build should be influenced by the first funded design partner.

38. FYP Aim

The academic aim of the project is:

To design, implement and evaluate an end-to-end AI engineering analytics platform that connects AI-assisted development activity with engineering delivery, software quality, customer-facing signals and organisational outcomes while preserving traceability, evidence quality, privacy and analytical validity.

39. FYP Objectives

The project should aim to:

1. Develop developer-side collection capabilities for selected AI coding tools.
2. Build integrations for selected engineering systems.

3. Define a provider-neutral canonical event model.
 4. Implement local privacy filtering and sanitisation.
 5. Build scalable event ingestion using Kafka or an equivalent justified mechanism.
 6. Implement analytical persistence using ClickHouse.
 7. Implement PostgreSQL-based control-plane services.
 8. Retain canonical archived data in S3-compatible storage.
 9. Build an evidence graph connecting AI activity to engineering and downstream outcomes.
 10. Implement the four AI evidence cohorts.
 11. Develop natural-grain metric definitions and valid aggregation rules.
 12. Implement a customer-configurable dimensional model.
 13. Provide explainable AI-versus-non-AI comparisons.
 14. Implement dashboard views such as Overview, Compare, Explore, Live Trace and Data Health.
 15. Test source identity, correlation accuracy and metric reproducibility.
 16. Validate the system progressively using synthetic data and Difinity data where available.
 17. Document architectural trade-offs and unresolved analytical limitations.
-

40. FYP Development Methodology

Development should follow an iterative approach in which each stage produces a testable artefact.

Phase 1: Requirements and Metric Definition

Define:

- target questions;
- entities;
- evidence states;
- connectors;
- metrics;
- source fields;
- privacy requirements.

Output:

requirements baseline and metric catalogue.

Phase 2: Architecture and Data Model

Define:

- event contracts;
- service boundaries;
- evidence graph;
- dimensional graph;
- storage architecture;
- tenant model;
- privacy boundaries.

Output:

justified architecture and shared data model.

Phase 3: Synthetic Data Foundation

Generate representative:

- AI sessions;
- commits;
- PRs;
- work items;
- deployments;
- defects;
- support records;
- outcomes.

Known relationships should be deliberately created so correlation logic can be tested objectively.

Output:

repeatable synthetic development and test dataset.

Phase 4: Collection and Connectors

Implement a limited set of:

- AI adapters;
- source-control integration;
- work-management integration;
- deployment integration.

Output:

working initial ingestion.

Phase 5: Pipeline and Storage

Implement:

- admission API;
- Kafka;
- event processing;
- ClickHouse;
- S3 archive;
- PostgreSQL control plane.

Output:

scalable end-to-end event flow.

Phase 6: Correlation and Evidence Graph

Implement rules relating:

- sessions;
- changes;
- commits;
- PRs;
- work items;
- releases;
- quality events;
- support signals;
- outcomes.

Each linkage should preserve evidence status and linkage method.

Output:

traceable correlated engineering dataset.

Phase 7: Metrics

Implement selected:

- spend metrics;
- AI coverage;
- delivery metrics;
- quality metrics;
- evidence-health metrics.

Output:

reproducible analytical metrics.

Phase 8: Dashboard Implementation

Build:

- Overview;
- Compare;
- Explore;
- Live Trace;

- Data Health.

Output:

working analytical interface.

Phase 9: Industry Validation

Apply the platform to Dfinity data where made available.

Use the findings to identify differences between initial assumptions and real workflows.

Output:

validation findings and architecture revisions.

Phase 10: Potential Design-Partner Validation

Where a suitable organisation is available, evaluate an end-to-end integration using one coherent engineering stack.

Output:

external usability and technical validation evidence.

41. Testing and Validation Strategy

41.1 Synthetic Validation

Use controlled synthetic scenarios to verify:

- schemas;
- correlation;
- event processing;
- metric calculations;
- attribution;
- roll-ups;
- dashboard output.

Known expected results should allow objective comparison.

41.2 Connector Validation

Compare ingested records with source-system data.

Validate:

- identifier preservation;
 - incremental synchronisation;
 - update behaviour;
 - mapping accuracy;
 - error handling.
-

41.3 Correlation Validation

Construct known session-to-change-to-PR-to-deployment relationships.

Determine whether Echo reconstructs them correctly.

Evaluation should consider both:

- correctness; and
 - explainability.
-

41.4 Metric Validation

Selected metrics should be independently recalculated outside the dashboard.

The manually or independently calculated values should match Echo's output.

41.5 Data-Health Validation

Test scenarios with:

- missing telemetry;
- incomplete linkage;
- stale sources;
- unresolved identities;
- conflicting hierarchy mappings;
- immature defect windows.

The platform should represent them as incomplete evidence rather than silently producing confident comparisons.

41.6 Difinity Data Validation

Where available, Difinity data should be used to determine:

- whether schemas fit real engineering workflows;

- whether expected mappings exist;
 - whether identity resolution works;
 - whether metrics remain meaningful;
 - whether data-health states correctly reveal gaps.
-

41.7 Architecture Review

Difinity representatives, including Wasim as described in the FYP proposal, can review and challenge:

- architectural decisions;
- technology choices;
- scalability assumptions;
- connector design;
- analytical assumptions;
- privacy decisions.

Accepted changes should be documented.

42. Technical Challenges

The hardest parts of Echo are systems and analytical problems rather than one isolated machine-learning algorithm.

Major challenges include:

Heterogeneous Data

Each system exposes different:

- identifiers;
- schemas;
- timestamps;
- update mechanisms;
- entity structures.

Reliable Correlation

Evidence from independent platforms must be linked without pretending uncertain relationships are confirmed.

Identity Resolution

The same developer may appear differently across:

- AI tools;
- Git;
- GitHub;

- Jira;
- organisation directories.

Evidence Completeness

Missing data must remain distinguishable from genuine absence of AI contribution.

Attribution

One engineering event may affect several:

- teams;
- initiatives;
- products;
- services.

Totals must not be unintentionally duplicated.

Metric Validity

Metrics must be calculated at their correct analytical grain.

Scalability

Event volume, connector count, and analytical query load may increase significantly.

Tenant Isolation

Cross-customer data leakage must be prevented at every layer.

Privacy

The platform must collect useful evidence without collecting source code, prompts, or other unnecessarily sensitive content.

Outcome Attribution

Customer and business metrics frequently operate at a much higher level than individual code contributions.

Echo must therefore distinguish evidence association from claims of causality.

43. Expected FYP Deliverables

The expected deliverables include:

- project requirements;
- formal analytics questions;
- architecture documentation;

- canonical event schema;
 - evidence graph;
 - customer-configurable dimensional model;
 - privacy and collection contract;
 - developer-side AI collection prototype;
 - selected engineering-system connectors;
 - event ingestion pipeline;
 - Kafka implementation;
 - ClickHouse analytics store;
 - S3-compatible archive;
 - PostgreSQL control plane;
 - correlation engine;
 - AI evidence cohort implementation;
 - metric catalogue;
 - aggregation rules;
 - dashboard application;
 - Live Trace functionality;
 - Data Health functionality;
 - synthetic datasets;
 - repeatable validation scenarios;
 - connector-validation results;
 - metric-validation results;
 - industry-data findings;
 - architecture trade-off documentation;
 - final technical report.
-

44. Expected Outcome

The expected outcome of the FYP is a **working prototype demonstrating how AI-assisted software engineering can be measured through a connected and explainable evidence model.**

A successful prototype should allow a user to move from a high-level observation such as:

AI-linked work has a lower median cycle time than comparable capture-complete no-AI-observed work

to the underlying evidence that produced the result.

The platform should allow inspection of:

- eligible work items;
- AI cohort definition;
- sample size;
- underlying pull requests;
- associated AI evidence;
- relevant deployments;
- quality windows;
- data-health status.

This traceability is central to Echo's value.

45. Success Criteria

Ingestion

Selected source and AI events reliably enter the platform in a repeatable form.

Data Model

Core facts and relationships are represented while retaining original source identity.

Privacy

Prohibited content is removed locally and does not enter the cloud ingestion path.

Correlation

Known engineering relationships can be reconstructed accurately and explainably.

AI Cohorts

AI-linked, AI-exposed-only, capture-complete no-AI-observed, and unknown work remain analytically separate.

Metrics

Selected measures are reproducible from documented definitions.

Aggregation

Roll-ups follow the declared natural-grain and deduplication rules.

Dashboard

Users can move from aggregate comparisons to supporting evidence.

Data Health

The system exposes missing, stale, conflicted or immature evidence rather than silently treating it as valid.

Scalability

The architecture allows increases in data volume and connector count without redesigning the entire platform.

Validation

Synthetic tests are successful, Difinity data is evaluated where accessible, and external feedback is documented where design-partner access is available.

46. Confirmed Product and Architecture Decisions

The following should currently be treated as the strongest product decisions.

- The product is named **Echo**, although exact brand treatment remains open.
 - Echo is a continuous AI engineering analytics platform.
 - ROI is a marketing promise rather than a standalone board or investment workflow.
 - Echo should operate independently from Hub and Flow.
 - Hub and Flow may provide enrichment data.
 - Claude Code, Codex, GitHub Copilot and Cursor are the first intended AI collectors.
 - The platform uses a metadata-only default collection model.
 - Sanitisation occurs on the endpoint before outbound persistence.
 - AI contribution is divided into four evidence cohorts.
 - A customer-configured dimensional graph is preferred over a fixed organisational hierarchy.
 - Metric calculations occur at their natural analytical grain.
 - Ownership, contribution, attributed and touched semantics remain explicit.
 - Kafka remains the event-streaming and durable-log layer.
 - ClickHouse is the preferred hot analytical database.
 - S3-compatible storage is the canonical archive.
 - PostgreSQL is the mutable control plane.
 - SaaS, customer-cloud and private deployments are platform requirements.
 - The Overview should use a consolidated comparison matrix.
 - Developer and session activity are diagnostic rather than productivity scores.
 - No composite magic productivity score is required.
 - No generated executive narrative is currently required.
-

47. Current Working Directions Rather Than Final Decisions

The following are strong working directions but should not yet be described as fully implemented or customer validated:

- customer-configurable dimensional graph;

- consolidated comparison matrix;
 - configurable primary metric for each analytical family;
 - Overview, Compare, Explore, Live Trace, Data Health and Data Policy/Configuration pages;
 - detailed owned/contributed semantics;
 - attributed versus touched measurements;
 - metric maturity rules;
 - customer-specific hierarchy imports.
-

48. Major Open Decisions

Important unresolved questions include:

1. What minimum dimension families are necessary for v1?
 2. Which relationship types are necessary for v1?
 3. Should the customer dimensional model permit full DAG behaviour?
 4. How should graph versioning work?
 5. How should cycle prevention work?
 6. Which source should initially seed organisational structure?
 7. What should the default ownership rule be?
 8. What should the default contribution rule be?
 9. What should the default cost-allocation policy be?
 10. Should non-financial events be fractionally allocated?
 11. What is the primary Overview metric for each analytical family?
 12. How should heterogeneous customer outcomes be summarised?
 13. Should individual developer views be restricted?
 14. How should mapping disputes and corrections be handled?
 15. What minimum evidence completeness is needed before a comparison is shown?
 16. Which funded design partner will determine the first integration stack?
 17. What are the formal throughput and latency targets?
 18. What retention periods will apply?
 19. How will ClickHouse perform on production-shaped Echo workloads?
 20. What tenant-isolation levels will be commercially offered?
 21. What is the default pseudonymity policy?
 22. What is the product pricing model?
 23. Will Echo formally become a third Dfinity product beside Hub and Flow?
-

49. Current Evidence and Validation Status

The current project must be described accurately.

At present:

- no customer outcome has been proven by the current prototype;
- no Echo endpoint collector is production-proven;
- no complete customer connector stack has been validated end-to-end;
- no production-scale ClickHouse benchmark exists for Echo-shaped data;
- no SaaS deployment has yet been proven at production scale;

- no customer-owned-cloud deployment has yet been proven;
 - no private-data-centre deployment has yet been proven;
 - no air-gapped deployment has yet been proven;
 - the product category still requires buyer validation;
 - the information architecture still requires buyer validation;
 - the dimensional graph is a working product direction;
 - the comparison matrix is a working product direction;
 - any prototype numbers are illustrative and must not be represented as customer results.
-

50. Scope of Proposed Research Extensions

The original FYP proposal additionally discusses three potential novelty areas:

1. productivity prediction using machine-learning models;
2. an LLM-based engineering analytics assistant;
3. AI governance integration.

These ideas remain potentially useful, but they should be treated as **optional extensions rather than the core Echo definition**.

The current product brief gives greater priority to:

- evidence quality;
- cohort comparison;
- correct aggregation;
- traceability;
- privacy;
- configurable organisation models;
- integration quality;
- data health.

Therefore, predictive ML or an LLM assistant should only be added once the underlying evidence and metric platform is sufficiently robust.

AI governance integration could be particularly compatible with Difinity because Echo can be enriched using information from Hub and Flow, but this should not replace Echo's central purpose as an AI engineering analytics platform.

51. Recommended FYP Core Scope

For a realistic Final Year Project, the strongest implementation would focus on a narrow but complete vertical slice.

A suitable scope would be:

AI Collection

At least one or two of:

- Claude Code
- OpenAI Codex
- GitHub Copilot
- Cursor

Work Management

One of:

- Jira
- Linear

Source Control

GitHub.

Deployment

GitHub Actions and GitHub Deployments.

Quality

Sentry, if implementation time permits.

Analytics Infrastructure

- Kafka
- ClickHouse
- S3-compatible object storage
- PostgreSQL

Product Features

- AI evidence cohort classification;
- evidence graph;
- basic configurable dimensions;
- Overview;
- Compare;
- Live Trace;
- Data Health.

This would demonstrate the core Echo thesis much more convincingly than implementing many shallow connectors.

52. Consolidated System View

The overall system can be summarised as:

AI tools + provider APIs + engineering systems

↓

Vendor-specific connectors and collectors

↓

Local privacy filtering and canonical event normalisation

↓

Git and identity provenance

↓

Authenticated Echo admission API

↓

Kafka durable event log

↓

Processing, normalisation, enrichment and correlation

↓

ClickHouse hot analytics

plus

S3 canonical archive

and

PostgreSQL control plane

↓

Evidence graph + natural-grain metric engine

↓

AI-linked versus capture-complete no-AI-observed comparison



Overview / Compare / Explore / Live Trace / Data Health



Engineering leadership decision support

53. Final Project Positioning

Difinity Echo should ultimately be positioned not as a system that attempts to prove:

“AI makes developers X% more productive.”

Instead, it provides the infrastructure and analytical controls necessary to answer more defensible questions such as:

“Within this defined engineering scope, using these evidence criteria and this observation window, how did AI-linked work differ from comparable capture-complete work in delivery speed, review effort, quality, support demand, expenditure, and product outcomes?”

This distinction is important.

The value of Echo is therefore based on five fundamental properties:

Traceability — high-level results can be traced back to engineering evidence.

Comparability — AI-linked work is compared with an analytically valid comparator.

Evidence awareness — incomplete data is represented explicitly rather than hidden.

Analytical validity — metrics are calculated at their natural grain and aggregated correctly.

Privacy — useful engineering telemetry is collected without default collection of prompts, source code, transcripts, or other unnecessary sensitive content.

Together, these properties allow Echo to provide a more credible view of AI engineering performance and ROI than isolated provider usage statistics or generic engineering dashboards.